

AUTOMATIC TOUCHLESS WATER DISPENSER

An Ultrasonic-Sensor-Based Hands-Free Water Dispensing System

Prepared by: Adu-Gyamfi Kwadwo

B.Sc. Electrical/Electronic Engineering, Kwame Nkrumah University of Science and Technology (KNUST)

Group Project — Third Year, 2023 (Reconstructed Documentation)

August 2026

Abstract

Manually operated taps and dispensers require direct hand contact, which can spread germs and encourage cross-contamination in shared spaces such as offices, schools, hospitals, and homes. This report documents the design and implementation of an Automatic Touchless Water Dispenser built around an Arduino Uno microcontroller. An HC-SR04 ultrasonic sensor continuously measures the distance to any object placed in front of it. When a hand or container is detected within a set range, the Arduino switches a relay that powers a small DC water pump, releasing water; when the object is withdrawn, the flow stops automatically. The system was originally built and demonstrated as a third-year group hardware project. This document reconstructs the project's design, wiring, and firmware in a more rigorous and professional form, situates it against similar systems reported in recent literature, and outlines a path for extending it into a small embedded-systems research project (e.g., adding volume measurement, IoT connectivity, or adaptive detection thresholds).

1. Introduction

Hygiene-conscious automation has become increasingly important in shared public and domestic spaces. A conventional tap or dispenser is touched by every user, which can transfer bacteria and viruses between people. A touchless dispenser removes this point of contact: a user simply places a hand or cup close to the device, and water is released without anyone needing to press a button or turn a handle.

This project uses low-cost, widely available components — an Arduino Uno, an HC-SR04 ultrasonic distance sensor, a relay module, and a small DC water pump — to build a working touchless dispenser. The choice of an ultrasonic sensor (rather than, for example, an infrared proximity sensor) was made because the HC-SR04 gives an actual distance reading in centimeters, which makes the activation range easy to tune and easy to explain and justify in a report.

2. Related Work

Several recent student and hobbyist projects report similar touchless dispenser designs built around the same core components used here. A DIY build documented by Electronics For You uses an Arduino board with an HC-SR04 sensor, a BC547 transistor to drive a 5V relay, and a small water pump, and reports that the relay is energized when a user's hand is roughly 3–5 cm from the sensor.

Recent undergraduate papers extend the same basic idea. One 2026 paper describes a system that uses an Arduino Uno, an HC-SR04 sensor, and an L298N motor driver (instead of a simple relay) to first dispense a fixed volume of about 100 mL and then switch to continuous flow if the object remains in place, with an LCD giving the user real-time status feedback. Another 2026 study on a similar Arduino Uno and HC-SR04 design reports that the ultrasonic sensor achieved 100% detection accuracy at 5 cm and about 95% accuracy at 10 cm, with a relay/pump response success rate of about 90% and an average response time close to 1.5 seconds — figures that are useful as a benchmark for evaluating this project's own performance once it is rebuilt in hardware. A separate 2026 paper on the same class of system reports that careful code optimization reduced the pump's shut-off response time from about 5.2 seconds down to roughly 0.02 seconds, which highlights that the software debounce and control logic (not just the sensor hardware) has a large effect on how responsive and water-efficient the final system feels.

Taken together, this body of related work confirms that the Arduino Uno plus HC-SR04 plus relay/pump architecture is a well-established and reproducible approach, while also pointing to concrete ways this project can be strengthened: adding a defined dispense volume or timeout, adding user feedback (LED or LCD), and paying close attention to software response time rather than relying on the sensor and relay defaults alone.

3. Objectives

- Detect the presence of a hand, cup, or bottle within a defined activation range (target: 2–10 cm) using an ultrasonic sensor.
- Automatically switch a water pump ON while an object remains in range, and OFF as soon as it is removed.
- Avoid false triggers from noisy sensor readings using simple software debouncing.
- Include a maximum run-time safety cutoff so the pump cannot run indefinitely if an object is left in place or the sensor misreads.
- Produce wiring documentation and firmware detailed enough that the system can be rebuilt in hardware and demonstrated to faculty.

4. System Design and Methodology

4.1 System Overview

The system follows a simple sense–decide–act loop. The HC-SR04 sensor (the 'sense' stage) repeatedly measures distance. The Arduino Uno (the 'decide' stage) compares each reading against the activation range and applies a short debounce check. The relay and pump (the 'act' stage) then turn the water flow on or off. Figure 1 shows the wiring between all components.

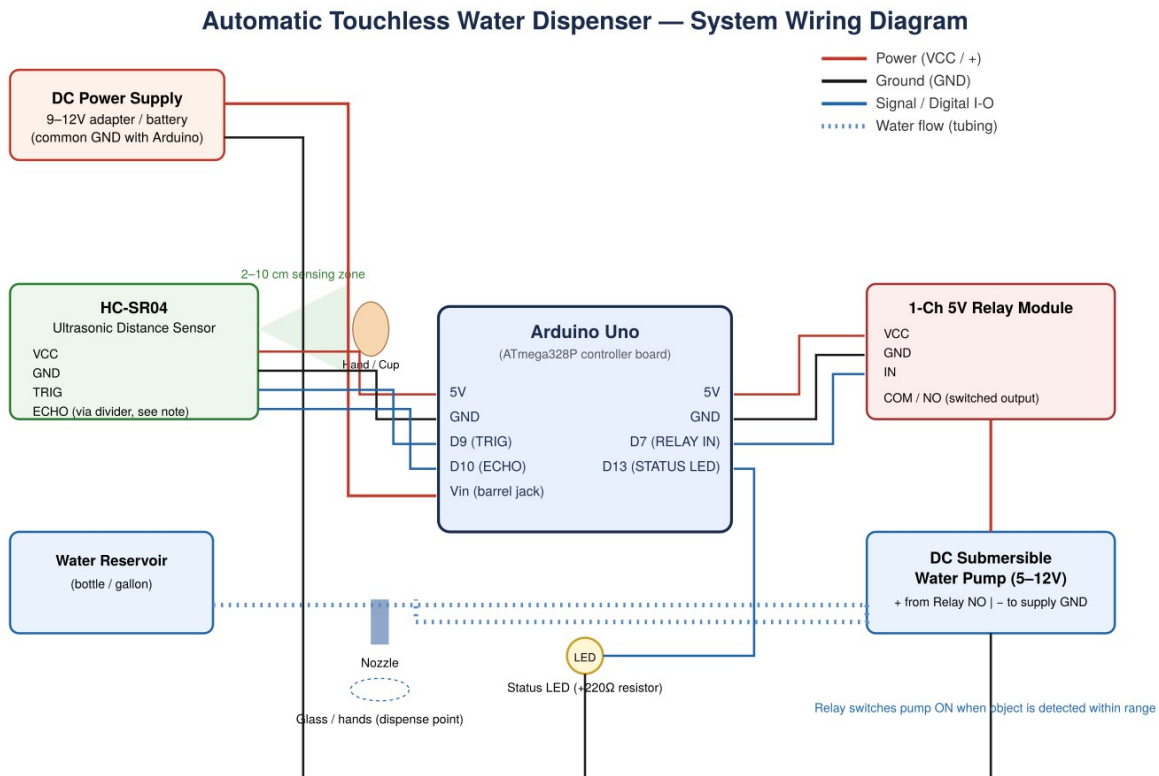


Figure 1. System wiring / block diagram of the Automatic Touchless Water Dispenser.

4.2 Original Prototype (2023)

The photographs below are stills from a short video recording of the original hardware, built and demonstrated as a third-year group project. The enclosure was constructed from cardboard as a low-cost proof-of-concept housing, with the ultrasonic sensor's two transducers visible on the front face (resembling a pair of "eyes"). These images are included as evidence that the design in this report was physically built and functioned as described, even though the original hardware and files are no longer available.



Figure 2a. Cardboard enclosure with the ultrasonic sensor mounted on the front face.



Figure 2b. The dispenser releasing water into a cup after the sensor detects it.

4.3 Bill of Materials

Component	Quantity	Purpose
Arduino Uno (ATmega328P)	1	Main controller: reads the sensor and drives the relay
HC-SR04 Ultrasonic Sensor	1	Measures distance to detect a hand/cup
1-Channel 5V Relay Module	1	Switches the pump's higher-current circuit using a low-current Arduino signal
DC Submersible Water Pump (5–12V)	1	Moves water from the reservoir to the nozzle
Silicone/PVC tubing	~0.5 m	Carries water from pump to nozzle
Water reservoir (bottle/gallon)	1	Water source
9–12V DC power adapter or battery pack	1	Powers the pump (and optionally the Arduino via Vin)

Component	Quantity	Purpose
LED + 220Ω resistor	1 each	Optional status indicator (pump ON/OFF)
Jumper wires / breadboard or perfboard	As needed	Connections and prototyping

4.4 Pin Connections

From	To (Arduino Uno)	Wire Type
HC-SR04 VCC	5V	Power
HC-SR04 GND	GND	Ground
HC-SR04 TRIG	Digital Pin 9 (output)	Signal
HC-SR04 ECHO	Digital Pin 10 (input)	Signal
Relay VCC	5V	Power
Relay GND	GND	Ground
Relay IN	Digital Pin 7 (output)	Signal
Relay COM / NO	External 5–12V supply + Pump (+)	Switched power (external supply, not Arduino)
Status LED (+220Ω)	Digital Pin 13	Signal

The pump and its higher-current supply are wired through the relay's switched contacts, not directly through the Arduino. This protects the Arduino's output pins, which are only rated for small currents (well under what a pump motor draws).

5. Arduino Firmware

The firmware below implements the sense–decide–act loop described in Section 4.1, with two additions beyond the simplest possible version of this project: (1) software debouncing, which requires several consecutive in-range readings before turning the pump on, so a single noisy reading cannot falsely trigger it; and (2) a maximum run-time safety cutoff, which forces the pump off after a fixed number of seconds even if an object is left in front of the sensor. These two additions directly respond to the response-time and reliability issues raised in the related work reviewed in Section 2.

```

/*
 * =====
 * Automatic Touchless Water Dispenser
 * -----
 * Description:
 *   A hands-free water dispensing system built on an Arduino Uno. An HC-SR04
 *   ultrasonic sensor continuously measures the distance to whatever is in
 *   front of it. When an object (a hand, cup, or bottle) is detected inside
 *   a defined activation range, the Arduino energizes a relay that switches
 *   a DC water pump ON. When the object is removed, the pump switches OFF.
 * Hardware:
 *   - Arduino Uno (ATmega328P)
 *   - HC-SR04 Ultrasonic Distance Sensor
 *   - 1-Channel 5V Relay Module (driving a DC submersible water pump)
 *   - Status LED (optional, on-board or external)
 * Pin Map:

```

```

*   TRIG_PIN   -> D9    (Arduino output -> HC-SR04 TRIG)
*   ECHO_PIN   -> D10   (HC-SR04 ECHO   -> Arduino input)
*   RELAY_PIN  -> D7    (Arduino output -> Relay IN)
*   STATUS_LED -> D13   (Arduino output -> indicator LED)
*
*   Safety / design notes:
*   - The HC-SR04 ECHO line is 5V logic. If the relay module or pump draws
*     current on the same 5V rail as a 3.3V-logic board, a voltage divider
*     would be needed on ECHO. On the Arduino Uno (5V logic) this is NOT
*     required.
*   - A maximum run-time cutoff (MAX_DISPENSE_TIME_MS) is included so the
*     pump cannot be left running indefinitely if an object is left in
*     front of the sensor (e.g., a cup placed and forgotten, or a sensor
*     fault). This is a basic safety/water-conservation feature that most
*     hobbyist versions of this project skip.
*   - Simple software debouncing (CONFIRM_READINGS) avoids false triggers
*     from a single noisy ultrasonic reading.
*
*   Author: Adu-Gyamfi Kwadwo
*   Course/Project: Third-Year Group Project - Electrical/Electronic Engineering
*   =====
*/

// ----- Pin definitions -----
const uint8_t TRIG_PIN   = 9;
const uint8_t ECHO_PIN   = 10;
const uint8_t RELAY_PIN  = 7;
const uint8_t STATUS_LED = 13;

// ----- Tunable parameters -----
const float   ACTIVATION_DISTANCE_CM = 10.0; // trigger if object is closer than this
const float   MIN_VALID_DISTANCE_CM  = 2.0;  // ignore readings below sensor's reliable
range
const uint8_t CONFIRM_READINGS       = 3;     // consecutive detections needed before
turning pump ON
const uint16_t SAMPLE_INTERVAL_MS    = 60;    // time between ultrasonic pings
const uint32_t MAX_DISPENSE_TIME_MS  = 8000UL; // safety cutoff: max continuous pump run-
time
const uint32_t COOLDOWN_AFTER_CUTOFF_MS = 1500UL; // forced pause after a safety cutoff

// Set to true if the relay module is "active LOW" (most cheap 1-channel
// relay boards energize when IN is pulled LOW). Set to false for
// "active HIGH" boards. Check your specific module's datasheet/silkscreen.
const bool RELAY_ACTIVE_LOW = true;

// ----- State variables -----
uint8_t consecutiveDetections = 0;
bool    pumpIsOn              = false;
uint32_t pumpStartTime         = 0;
bool    inCooldown            = false;
uint32_t cooldownStartTime     = 0;
uint32_t lastSampleTime        = 0;

// =====
void setup() {
    Serial.begin(9600);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(RELAY_PIN, OUTPUT);
    pinMode(STATUS_LED, OUTPUT);

    setPump(false); // ensure pump starts OFF
    digitalWrite(STATUS_LED, LOW);

    Serial.println(F("Touchless Water Dispenser - system initialized.));
}

// =====

```

```

void loop() {
    uint32_t now = millis();

    // Handle a forced cooldown period after a safety cutoff
    if (inCooldown) {
        if (now - cooldownStartTime >= COOLDOWN_AFTER_CUTOFF_MS) {
            inCooldown = false;
            consecutiveDetections = 0;
        } else {
            return; // skip sensing/dispensing while cooling down
        }
    }

    // Take a distance reading no more often than SAMPLE_INTERVAL_MS
    if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) {
        lastSampleTime = now;
        float distance = readDistanceCM();
        handleDetection(distance, now);
    }

    // Enforce the maximum continuous run-time safety cutoff
    if (pumpIsOn && (now - pumpStartTime >= MAX_DISPENSE_TIME_MS)) {
        Serial.println(F("Safety cutoff: max dispense time reached. Stopping pump."));
        setPump(false);
        inCooldown = true;
        cooldownStartTime = now;
    }
}

// =====
// Sends a 10us pulse on TRIG and measures the ECHO pulse width to compute
// distance in centimeters. Returns -1.0 if no valid echo was received
// (i.e., nothing in range, or a sensor timeout).
float readDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    // 25ms timeout ≈ 4.3 m maximum range; prevents pulseIn() from blocking forever
    uint32_t durationUs = pulseIn(ECHO_PIN, HIGH, 25000UL);

    if (durationUs == 0) {
        return -1.0; // no echo received (out of range / timeout)
    }

    // Speed of sound ≈ 343 m/s at room temperature -> 0.0343 cm/us.
    // Divide by 2 because the pulse travels to the object and back.
    float distanceCm = (durationUs * 0.0343f) / 2.0f;
    return distanceCm;
}

// =====
// Applies the debounced activation logic: an object must be detected within
// range for CONFIRM_READINGS consecutive samples before the pump turns on,
// and the pump turns off as soon as a single "no object" reading occurs.
void handleDetection(float distanceCm, uint32_t now) {
    bool objectPresent = (distanceCm > MIN_VALID_DISTANCE_CM) &&
        (distanceCm <= ACTIVATION_DISTANCE_CM);

    if (objectPresent) {
        consecutiveDetections++;
        Serial.print(F("Distance: "));
        Serial.print(distanceCm);
        Serial.println(F(" cm [object in range]"));

        if (consecutiveDetections >= CONFIRM_READINGS && !pumpIsOn) {
            setPump(true);
        }
    }
}

```

```

        pumpStartTime = now;
    }
} else {
    consecutiveDetections = 0;
    if (pumpIsOn) {
        setPump(false);
    }
}
}

// =====
// Turns the pump (relay) and status LED on/off, honoring the relay's
// active-HIGH / active-LOW wiring.
void setPump(bool turnOn) {
    pumpIsOn = turnOn;
    bool relaySignal = RELAY_ACTIVE_LOW ? !turnOn : turnOn;
    digitalWrite(RELAY_PIN, relaySignal ? HIGH : LOW);
    digitalWrite(STATUS_LED, turnOn ? HIGH : LOW);

    Serial.println(turnOn ? F(">>> Pump ON") : F(">>> Pump OFF"));
}

```

6. Working Principle

- The Arduino sends a short 10-microsecond pulse on the TRIG pin, causing the HC-SR04 to emit an ultrasonic burst.
- The sensor's ECHO pin goes HIGH for a duration proportional to the time the pulse takes to bounce back from the nearest object.
- The Arduino converts this time into a distance in centimeters using the speed of sound.
- If the distance stays within the activation range (2–10 cm) for three consecutive readings, the Arduino drives the relay pin, which energizes the relay and switches the pump on.
- As soon as a reading falls outside the activation range, the pump is switched off immediately.
- If the pump has been running continuously for more than 8 seconds (the safety cutoff), it is forced off and the system pauses briefly before it will respond to new detections again.

7. Testing Plan (Software Stage)

The original hardware build is not currently available, so this stage of the work is being carried out and verified in software before the physical parts are reassembled. The firmware in Section 5 can be tested in two ways without hardware in hand:

- **Simulation:** Load the sketch into a simulator such as Wokwi or Tinkercad Circuits, which both support the Arduino Uno and the HC-SR04 sensor. Vary the simulated sensor distance and confirm the relay/pump output and Serial Monitor readings respond as described in Section 6.
- **Serial Monitor dry-run:** Even without a simulator, the code can be uploaded to any spare Arduino Uno with only the HC-SR04 and an LED connected, and the printed distance readings and pump ON/OFF messages checked directly against hand movement in front of the sensor.

Once the pump, relay, and tubing are back in hand, the same test should be repeated physically: verifying the activation distance, checking that flow starts and stops promptly, and timing the response to compare against the benchmark figures from Section 2 (for example, the ~1.5 second average response time and the 90%+ success rate reported in comparable published designs).

8. Applications and Future Work

This kind of touchless dispenser is directly useful in kitchens, offices, schools, clinics, and public restrooms, anywhere reducing surface contact is valuable. Beyond the working prototype, the design in this report leaves room for several extensions that would turn it from a hobby build into a small applied-research project:

- Metered dispensing: add a flow sensor so the system dispenses an exact volume (e.g., 100 mL) rather than running for as long as an object is detected, similar to the L298N-driven design discussed in Section 2.
- User feedback: add a small LCD or a buzzer to confirm detection and dispensing status to the user.
- Connectivity: replace the Arduino Uno with an ESP32/ESP8266 to log usage data (litres dispensed per day, peak usage times) to the cloud, enabling a small IoT/data-analysis study.
- Adaptive thresholding: instead of a fixed 2–10 cm activation range, use a short calibration routine at startup so the system adapts to its mounting height and environment automatically.
- Energy source: evaluate a solar-charged battery supply for off-grid or low-infrastructure deployment.

9. Conclusion

This report reconstructs the design, wiring, and firmware of an Arduino Uno-based touchless water dispenser originally built as a third-year group project. The system uses an HC-SR04 ultrasonic sensor to detect a nearby hand or container and a relay-driven DC pump to release water automatically, with software debouncing and a safety run-time cutoff added to improve reliability. Comparing the design against recently published student projects on the same topic confirms that the underlying architecture is sound and reproducible, while also pointing to concrete next steps — metered dispensing, user feedback, connectivity, and adaptive calibration — that would extend this from a class project into a small embedded-systems research effort.

References

Electronics For You. Touchless Arduino Water Dispenser | Full DIY Project.

<https://www.electronicsforu.com/electronics-projects/hardware-diy/touchless-arduino-water-dispenser>

Fabro, E. B. C., Morabe, E. Z., Manales, E. J., Santuyo, I. E. B., Tuazon, N. M. C., & Villalobos, J. E. (2026). Touchless Water Dispensing System. *International Journal of Research and Scientific Innovation*, XIII(1), 425–437.

(2026). Design and Implementation of an Arduino Uno Based Smart Automatic Water Dispenser Prototype. *Jurnal Pepadun*.

(2026). Automatic Dispenser Using Distance Sensor. *Jurnal Teknisi*.

(2024). Automatic Water Dispenser: An IoT-enabled Water Dispenser with Touch-free Dispensing and App Control. *International Research Journal in Emerging Technology*.

Smart Water Dispenser Using Arduino. *International Journal of Research and Analytical Reviews (IJRAW)*.